1 1A ./test_objects output showing all tests passing

```
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ nano object.c
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1 1B find .pes/objects -type f showing sharded directory structure

```
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$
```

2 2A ./test_tree output showing all tests passing 2

```
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ nano tree.c
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ make clean
make test_tree
./test_tree
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

3 3A pes init → pes add → pes status sequence

```
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  Makefile
  untracked:  index.h
  untracked:  tree.c
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  tree.h
  untracked:  object.c
  untracked:  commit.c
  untracked:  pes.h
  untracked:  .gitignore
```

3 3B cat .pes/index showing the text-format index 4

```
poorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ cat .pes/index
00664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776330959 6 file1.txt
00664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776330959 6 file2.txt
poorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$
```

4 4A pes log output with three commits

```
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Committed: e5b39fb82701... Initial commit
Committed: d78b6cd06549... Update file
commit d78b6cd0654937912bc9b2d55329999f240fb6111b0043271f1b9a41d3d3720d
Author: PES User <pes@localhost>
Date:   1776332267

    Update file

commit e5b39fb827016994402ad909fc14fa167b841b961944817ea25c6b88e966be97
Author: PES User <pes@localhost>
Date:   1776332267

    Initial commit
```

4 4B find .pes -type f | sort showing object growth

```
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/d7/8b6cd0654937912bc9b2d55329999f240fb6111b0043271f1b9a41d3d3720d
.pes/objects/e5/b39fb827016994402ad909fc14fa167b841b961944817ea25c6b88e966be97
.pes/refs/heads/main
```

4 4C cat .pes/refs/heads/main and cat .pes/HEAD

```
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
d78b6cd0654937912bc9b2d55329999f240fb6111b0043271f1b9a41d3d3720d
ref: refs/heads/main
```

Final -- Full integration test (make test-integration)

```
spoorthi@spoorthi-VirtualBox:~/PES1UG24CS629-pes-vcs$ make test integration
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
=== Running Phase 1 tests ===
./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bb
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.

=== Running Phase 2 tests ===
./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===
```

Phase 5&6

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

pes checkout <branch>

To implement pes checkout <branch>, the following steps are required:

- Update the .pes/HEAD file to point to the selected branch:

ref: refs/heads/<branch>

- Read the commit hash from .pes/refs/heads/<branch>
- Load the corresponding commit and its tree object
- Update the working directory to match the tree:
  - Create new files
  - Update existing files
  - Delete files not present in the target tree

Complexity:

This operation is complex because it must handle uncommitted changes, file conflicts, and ensure no data loss occurs while modifying the working directory.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Detecting Dirty Working Directory

To detect uncommitted changes:

- For each file in the index:
  - Read the file from the working directory
  - Compute its hash
  - Compare it with the stored hash in the index
- If the hashes differ, the file is modified
- If a file is missing, it is deleted
- If a new file exists, it is untracked

Conflict Detection:

If a file is modified in the working directory and also differs in the target branch, the checkout operation must be refused to prevent overwriting changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Detached HEAD

A detached HEAD state occurs when .pes/HEAD contains a commit hash instead of a branch reference.

- Commits can still be created in this state
- However, no branch points to these commits
- These commits may become unreachable later

Recovery:

The user can recover these commits by creating a new branch pointing to the commit hash or by using the commit ID from logs.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Garbage Collection Algorithm

To remove unreachable objects:

1. Start from all branch heads in .pes/refs/heads/
2. Traverse all reachable objects:

o Commit → parent commits

o Commit → tree

o Tree → blobs and subtrees

3. Mark all visited objects
4. Delete objects not marked as reachable

Data Structure:

A hash set (hash table) is used to efficiently track visited objects.

Estimate:

For a repository with 100,000 commits and 50 branches, approximately 200,000 - 500,000 objects may need to be visited.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Garbage Collection Race Condition

Running garbage collection during a commit is dangerous due to race conditions.

- A commit may create new objects that are not yet referenced

- If GC runs at this time, it may delete these objects
- This results in broken commits referencing missing data

How Git Avoids This:

- Uses locking mechanisms to prevent concurrent operations
- Runs GC only when the repository is stable
- Uses atomic file operations (temporary files and rename)
- Ensures objects are fully written before being referenced